

The Compositional Depth Decay is a Dispatch Tax

Separating Parsing from Evaluation Collapses the Exponential Decay Rate α to Zero

Juan Cruz Dovzak

April 2026

Abstract

We study compositional depth extrapolation on balanced reverse-Polish modular arithmetic and identify a two-parameter empirical law for out-of-distribution accuracy: $\text{acc}(d) = \text{floor}_{\text{arch}} + (\text{acc}(d_{\text{min}}) - \text{floor}_{\text{arch}}) \cdot \exp(-\alpha_{\text{arch}}(d - d_{\text{min}}))$, where α_{arch} and $\text{floor}_{\text{arch}}$ fingerprint each architecture. Across 42 runs, leave-one-out prediction of unseen depths has mean absolute error 1.7–3.2 percentage points. We propose that α measures a *dispatch tax*: the per-depth cost of learning *when* to combine intermediate states during forward pass. Architectures that must infer execution structure pay this tax; architectures whose computational graph encodes it directly do not. We design **TreeMoTE**, a recursive shared-cell architecture whose forward pass traverses the expression parse tree, and test the extreme-case prediction $\alpha \rightarrow 0$. TreeMoTE achieves 100% accuracy at $p = 7$ depths up to $d = 8$ (OOD), extends to $d = 12$ at 93–100%, and outperforms Transformer by 80pp at $d = 8$. We validate mechanistically: the trained cell reproduces the complete $\mathbb{Z}/7\mathbb{Z}$ operator table (147/147 triples) and transfers shape-agnostically: trained on balanced RPN, applied without retraining to left-skewed linear RPN, the same cell achieves 100% at $d \leq 4$ and 93% at $d = 8$. At $p = 31$, where the operator table is too large for our training budget, TreeMoTE’s cell does not fully converge; however, the architectural advantage over Transformer persists ($5\times$ chance vs $1.1\times$ chance at $d = 5$ OOD). We do not claim that recursive or tree-augmented neural networks are novel—they are not. Our contribution is to (i) quantify depth-decay as an architectural property via α , (ii) identify dispatch discovery as the causal mechanism, (iii) demonstrate a causal intervention (TreeMoTE) that collapses α to zero when the parse is given, and (iv) decompose compositional-generalization failure into separable parsing, operator learning, and recursive-evaluation costs. We flag honestly that TreeMoTE requires an explicit parse and that cell learning remains a standard-ML problem that does not disappear at larger moduli.

1 Introduction

Neural architectures famously fail to extrapolate compositional depth [1, 2]. Given expressions of training depth 2, accuracy on depth 5 typically collapses to chance. The literature reports these failures as single-point accuracies and debates whether the limitation is of representation, optimization, or training data.

In this work we study modular reverse-Polish arithmetic as a controlled benchmark and approach the question mechanistically. We make four contributions:

1. **A two-parameter empirical law** (Section 3) describing out-of-distribution accuracy across depths, validated on 42 runs spanning three architectures and multiple curricula. Leave-one-out prediction for depths $\{6, 7, 8\}$ using a fit on $\{2, 3, 4, 5\}$ achieves mean absolute error of 1.7–3.2pp.
2. **The dispatch tax hypothesis** (Section 4). We propose that the decay rate α_{arch} is the per-depth cost of learning to dispatch—that is, of discovering *when* to combine intermediate states during forward pass. Architectures that must infer execution structure

from sequential tokens pay an accumulating cost per composition level; architectures that encode the structure directly should exhibit $\alpha \rightarrow 0$.

3. **TreeMoTE and a causal intervention** (Section 5). We design a recursive shared-cell architecture whose forward pass traverses the expression parse tree. TreeMoTE achieves 100% accuracy at test depths up to $d = 8$ (OOD) on $p = 7$, extends to $d = 12$ at 93–100%, and outperforms Transformer by 80 percentage points at $d = 8$. We verify the mechanism by (a) extracting the trained cell and confirming it reproduces the complete $\mathbb{Z}/7\mathbb{Z}$ operator table (147/147 triples), and (b) testing shape-transferability: a cell trained on balanced RPN evaluated on left-skewed linear RPN without retraining achieves 100% at $d \leq 4$ and 93% at $d = 8$.
4. **A decomposition of compositional-generalization failure** (Section 10). The result separates three components of the compositional-generalization problem—parsing, local-operator learning, and recursive evaluation—and identifies which TreeMoTE addresses and which it does not. At $p = 31$, TreeMoTE’s cell does not fully converge with our training budget, but the architectural advantage over Transformer persists by a factor of five over chance. Training on operands $\{0, 1, 2, 3\}$ and testing on $\{0, \dots, 6\}$ shows that value-space extrapolation remains a standard machine-learning problem, addressed by coverage and capacity rather than by tree structure.

We do not claim that recursive or tree-augmented neural networks are new. Tree-LSTMs [5], neural module networks [7], neural programmer-interpreters [8], differentiable stack machines [9, 10, 11], and tree stack memory units [12] all predate us. Our contribution is to isolate and measure the *dispatch tax* as a separable architectural cost, present a causal intervention that drives it to zero, and decompose the remaining failure modes.

2 Setup

Task. Balanced RPN on $\mathbb{Z}/p\mathbb{Z}$ with operators $\{+, -, \times\}$. A depth- d expression comprises $d+1$ operands followed by d operators: $[a_0, a_1, \dots, a_d, \text{op}_1, \dots, \text{op}_d]$. Evaluation is deterministic via a stack machine with max stack depth $d+1$. Labels are the final result modulo p .

Linear RPN (for shape-transfer test) interleaves operators: $[a_0, a_1, \text{op}_1, a_2, \text{op}_2, a_3, \text{op}_3, \dots]$. It evaluates left-foldedly with max stack depth 2. Its parse tree is left-skewed, whereas balanced RPN’s parse tree is right-skewed.

Null model. Modular multiplication collapses on zero operands, yielding an empirical label-0 frequency of ≈ 0.20 at $p = 7$. We rejection-sample label-uniform data; the true chance baseline is $1/p$. An earlier draft used the uniform baseline without correction and overstated architectural gaps by roughly $2\times$.

Architectures. We compare three flat/sequential baselines, all $\approx 450\text{K}$ parameters:

- **Transformer**, $d_{\text{model}}=128$, 2 layers, causal self-attention.
- **MoTE**, Transformer with a typed operator mixture of Op_{lin} and Op_{bil} .
- **StackMoTE (no-resid)**, differentiable soft stack with typed operators and no residual bypass.

For the causal-intervention experiment we introduce **TreeMoTE** (Section 5), a $\approx 65\text{K}$ -parameter architecture whose forward pass applies a single learned cell along the parse tree.

Training. WSD learning-rate schedule; AdamW with $\text{lr}=10^{-3}$, $\beta_1=0.9$, $\beta_2=0.98$, weight decay 1.0, gradient clip 1.0; batch size 256; WS 500 steps; 15K–50K total steps depending on experiment. Rejection-sampled label-balanced data throughout.

3 The two-parameter law

3.1 Single-depth training hides architectural differences

When models train on a single depth $d = 2$ balanced RPN, the ≈ 3000 -sequence training distribution is nearly fully memorized within 20K steps. All three flat architectures reach 58–70% on training and are statistically indistinguishable on out-of-distribution depths (Table 1). Architectural differences we document below are invisible in this regime.

Table 1: Single-depth training ($d_{\text{train}} = 2$), 20K steps.

Architecture	$d = 2$ (train)	$d = 3$	$d = 4$	$d = 5$
Transformer	0.664	0.192	0.148	0.148
MoTE	0.700	0.202	0.138	0.156
StackMoTE (soft)	0.580	0.178	0.192	0.184

3.2 Multi-depth curriculum exposes architectural separation

Under a curriculum spanning $d \in \{2, 3, 4, 5\}$, the space of $d = 3$ balanced-RPN expressions on $\mathbb{Z}/7\mathbb{Z}$ is $7^4 \times 3^3 \approx 65\text{K}$, vastly exceeding the 3000 samples seen per depth. The architectures must generalize even within training depths (Table 2).

Table 2: Multi-depth curriculum $\{2, 3, 4, 5\}$, balanced labels, three seeds.

Architecture	$d = 2$	$d = 3$	$d = 5$ (OOD)	$d = 8$ (OOD)
Transformer	0.42 ± 0.09	0.22 ± 0.01	0.17 ± 0.00	0.18 ± 0.01
MoTE	0.47 ± 0.12	0.24 ± 0.03	0.18 ± 0.02	0.17 ± 0.01
StackMoTE (no-resid)	0.61 ± 0.13	0.40 ± 0.04	0.28 ± 0.02	0.23 ± 0.01

3.3 The law

Across all architectures and curricula we study, the accuracy-vs-depth curve is well-described by

$$\text{acc}(d) = \text{floor}_{\text{arch}} + (\text{acc}(d_{\text{min}}) - \text{floor}_{\text{arch}}) \cdot e^{-\alpha_{\text{arch}}(d-d_{\text{min}})}. \quad (1)$$

The one-parameter form (floor = chance) underfits; allowing floor to be free improves the mean coefficient of determination from $R^2 = 0.795$ to 0.826 over 28 runs with ≥ 5 depth evaluations.

Table 3: Pooled parameters of Eq. 1, three seeds under curriculum $\{2, 3, 4, 5\}$.

Architecture	floor	α	Half-life (depth steps)
Transformer	0.155 ± 0.02	0.78 ± 0.18	0.89
MoTE	0.158 ± 0.01	0.76 ± 0.05	0.91
StackMoTE	0.220 ± 0.02	0.42 ± 0.09	1.66

3.4 Leave-one-out prediction

Fitting Eq. 1 on $d \in \{2, 3, 4, 5\}$ and predicting $d \in \{6, 7, 8\}$, across three architectures and three seeds, the mean absolute error of predicted accuracy is 1.7–3.2 percentage points, comparable to the sampling noise at 500 test examples per depth (Table 4).

Table 4: Leave-one-out prediction error: fit on $\{2, 3, 4, 5\}$, predict $\{6, 7, 8\}$.

Architecture	MAE $d=6$	MAE $d=7$	MAE $d=8$	Mean
Transformer	2.9pp	2.5pp	3.3pp	3.0pp
MoTE	1.2pp	0.7pp	3.4pp	1.9pp
StackMoTE	2.8pp	1.8pp	5.1pp	3.2pp

4 The dispatch tax hypothesis

We conjecture that α_{arch} is the log-cost per depth step of learning the *dispatch function* between intermediate states: the decision of when to combine operands versus when to retain them. In a stack machine, this is the push/pop controller; in a Transformer, it is the pattern of which tokens attention heads attend to.

Controller dispatch is discoverable. Probing StackMoTE’s three-way controller after training, digit tokens trigger *push* with probability 0.91 and operator tokens trigger *pop-apply* with probability 0.64, without any supervision. The correct dispatch function is learnable from causal language-modeling loss alone; its discovery is not binary but probabilistic and accumulates error with depth.

Dispatch entropy tracks α . Across three StackMoTE models trained on curricula $\{2, 3\}$, $\{2, 3, 4\}$, and $\{2, 3, 4, 5\}$, the summed per-layer dispatch entropy decreases monotonically ($H_{\text{sum}} = 1.11, 0.68, 0.44$), and the fitted α decreases in parallel (0.38, 0.33, 0.26). Broader training reduces controller uncertainty, which in turn reduces accumulated error with depth.

Extreme-case prediction. If α is the dispatch cost, an architecture whose computational graph directly encodes the parse tree—so that dispatch need not be learned at all—should exhibit $\alpha \rightarrow 0$.

5 TreeMoTE and the causal intervention

5.1 Architecture

TreeMoTE is a minimal recursive architecture: a single learned cell $\text{cell} : (\text{op}, \text{left}, \text{right}) \rightarrow \text{state}$, consisting of a typed mixture of a bilinear and a linear operator $\{\text{Op}_{\text{lin}}^{(\text{op})}, \text{Op}_{\text{bil}}^{(\text{op})}\}$ per operator index, and a softmax mix. The forward pass traverses the parse tree via a left-fold: starting from the rightmost operand embedding, it repeatedly applies the cell with the next operator and next-leftward operand embedding. The final state is projected through a tied classifier. TreeMoTE has no dispatch controller, no attention, and $\approx 65\text{K}$ parameters.

5.2 Empirical results at $p = 7$

Under curriculum $\{2, 3, 4, 5\}$ with balanced labels at $p = 7$:

Cross-entropy loss reaches 10^{-3} by step 5K; accuracy at all in-training and out-of-training depths equals 1.0 by step 15K.

Table 5: TreeMoTE vs baselines, seed 42, 20K steps. Chance = $1/7 \approx 0.143$.

Arch	$d = 2$	$d = 3$	$d = 4$	$d = 5$	$d = 6$	$d = 7$	$d = 8$
Transformer	0.472	0.270	0.244	0.232	0.194	0.166	0.174
StackMoTE (no-resid)	0.472	0.368	0.326	0.242	0.196	0.200	0.198
TreeMoTE	1.000	1.000	1.000	1.000	1.000	1.000	1.000

5.3 Multi-seed robustness

Across seeds $\{42, 123, 7\}$, TreeMoTE’s accuracy at $d = 8$ OOD is 0.994, 0.998, 0.984 (mean 0.992 ± 0.006).

5.4 Depth extrapolation to $d = 12$

Training on $d \in \{2, 3, 4, 5\}$ and evaluating up to $d = 12$:

Table 6: Depth extrapolation, seed 42, 20K steps.

Depth	$d = 5$	$d = 6$	$d = 7$	$d = 8$	$d = 9$	$d = 10$	$d = 11$	$d = 12$
TreeMoTE	1.000	0.998	1.000	0.996	0.986	0.980	0.944	0.960

Fitting Eq. 1 to TreeMoTE yields $\alpha \approx 0.01$ – 0.02 (Transformer under the same curriculum has $\alpha \approx 0.78$)—a 40–80 $\times$ reduction in decay rate. The residual decay at $d \geq 11$ reflects floating-point accumulation over recursive cell applications, not structural loss.

5.5 Matched-parameter comparison

At $d_{\text{model}} = 32$ (approximately 5 $\times$ fewer parameters than Transformer), TreeMoTE ($\approx 4\text{K}$ params) still dominates (Table 7).

Table 7: Matched-parameter test, $d_{\text{model}} = 32$, curriculum $\{2, 3, 4, 5\}$, 20K steps.

Arch	Params	$d = 2$	$d = 5$ (OOD)	$d = 7$ (OOD)	$d = 8$ (OOD)
Transformer	20K	0.318	0.204	0.172	0.190
StackMoTE	14K	0.214	0.204	0.180	0.198
TreeMoTE	4K	0.938	0.610	0.526	0.534

With five times fewer parameters, TreeMoTE outperforms Transformer by 34 percentage points at $d = 8$. The result is not driven by capacity.

6 Generalization beyond modular arithmetic: Boolean formulas

To test whether the principle survives outside modular arithmetic, we apply the same architectures to Boolean formula evaluation: vocabulary $\{0, 1, \text{AND}, \text{OR}, \text{XOR}\}$, balanced RPN, chance 50%.

TreeMoTE maintains 100% accuracy across all depths including OOD ($d \geq 6$). The principle transfers to a different task family.

Table 8: Boolean formulas, curriculum $\{2, 3, 4, 5\}$, 20K steps, seed 42.

Arch	$d = 2$	$d = 3$	$d = 4$	$d = 5$	$d = 6$	$d = 7$	$d = 8$
Transformer	1.000	0.992	0.980	0.952	0.718	0.742	0.706
StackMoTE (no-resid)	1.000	1.000	1.000	0.998	0.950	0.814	0.786
TreeMoTE	1.000	1.000	1.000	1.000	1.000	1.000	1.000

7 Ablation: typed operators vs recursive sharing

TreeMoTE combines two design choices: recursive parameter sharing and typed bilinear operators. We construct **TreeMLP**, which retains the recursive sharing but replaces the typed cell with a simple two-layer MLP on $[\text{left}; \text{right}; \text{onehot}(\text{op})]$, to isolate each contribution.

Boolean, $p = 7$: nearly no difference. On Boolean (12 operator triples total) and $p = 7$ (147 triples with full training coverage), TreeMLP matches TreeMoTE within 2–3pp at $d = 8$ (TreeMLP 97.2%, TreeMoTE 100% on $p = 7$; both 100% on Boolean). Recursive sharing is the dominant mechanism at full coverage.

Table 9: $p = 11$ balanced RPN, 3000 samples per depth ($= 25\%$ of 11979 unique operator triples), curriculum $\{2, 3, 4, 5\}$, 15K steps.

Arch	$d = 2$	$d = 3$	$d = 4$	$d = 5$	$d = 6$	$d = 7$	$d = 8$
Transformer	0.200	0.158	0.158	0.142	0.098	0.136	0.114
TreeMLP	0.288	0.220	0.204	0.176	0.122	0.154	0.172
TreeMoTE	1.000	1.000	1.000	1.000	0.998	0.996	0.992

$p = 11$, **25% operator-table coverage: TreeMoTE wins by 82pp.** At sparse coverage (25%), the typed bilinear cell is *dramatically more sample-efficient*: TreeMoTE achieves 99.2% at $d = 8$ while TreeMLP reaches 17.2%. The difference between TreeMoTE and TreeMLP—invisible at full coverage—emerges clearly at partial coverage.

Decomposition of the architectural contributions.

1. **Recursive parameter sharing + parse-given evaluation:** necessary for $\alpha \rightarrow 0$. Any tree-recursive cell suffices. This is the structural condition.
2. **Typed bilinear operators:** sufficient for sample-efficient operator learning. Materially matters when the operator table is large relative to training budget.

The two axes are orthogonal. Flat architectures fail on both.

8 Parse corruption: causal evidence

To confirm that TreeMoTE causally depends on the parse (rather than finding a parse-invariant representation), we shuffle operand positions at test time with increasing corruption rate c .

Under full corruption, accuracy at deep OOD depths collapses to chance ($d = 8$: 15.8% vs chance 14.3%). At $c = 0.5$, accuracy is roughly halfway between clean and chance. The model causally depends on the parse structure.

Table 10: Parse corruption on $p = 7$ trained TreeMoTE. At $c = 1.0$ operand positions are fully randomized; the parse the model expects is destroyed. Chance = 14.3%.

Test depth	$c = 0$	$c = 0.1$	$c = 0.25$	$c = 0.5$	$c = 1.0$
2	1.000	0.934	0.872	0.748	0.528
3	1.000	0.926	0.844	0.702	0.356
5	1.000	0.910	0.784	0.594	0.190
8	0.996	0.892	0.808	0.632	0.158

9 Mechanistic verification

9.1 The cell learns the operator table exactly

To confirm that the depth-OOD accuracy reflects a learned operator rather than a black-box policy, we extract the trained cell and evaluate it on all (a, b, op) triples:

Table 11: Cell table audit on the trained cell (seed 42, curriculum $\{2, 3, 4, 5\}$, 15K steps).

Operator	Correct	Total	Accuracy
+	49	49	1.000
$\times$	49	49	1.000
−	49	49	1.000
Total	147	147	1.000

The cell reproduces every $(a, b, \text{op}) \rightarrow (a \text{ op } b) \bmod 7$ triple exactly. The depth-OOD accuracy of 1.000 is therefore mechanistically explained: the cell implements the true arithmetic, and the architecture applies it recursively along the parse tree.

9.2 The cell transfers across tree shapes

To test that the learned cell is a pure operator function rather than specific to balanced-RPN structure, we apply the same trained cell, *without retraining*, to *linear* RPN (left-skewed parse trees):

Table 12: Shape-transfer bidirectional. 500 test examples per depth, no retraining across shapes.

Depth	1	2	3	4	5	6	7	8
Cell trained on balanced, tested on linear	1.000	1.000	1.000	1.000	0.994	0.974	0.956	0.928
Cell trained on linear, tested on balanced	1.000	0.998	0.996	0.982	0.948	0.930	0.890	0.792

Both directions transfer well: the cell trained in either tree shape reproduces the full 147/147 operator table (cell audit), and applies correctly to the other shape. At $d \leq 4$, accuracy is within 2pp of 100% in either direction; at $d = 8$ accuracy drops to 93% (balanced $\rightarrow$ linear) or 79% (linear $\rightarrow$ balanced), consistent with floating-point accumulation over longer recursive chains. The asymmetry is consistent with the cell’s training signal being slightly better-aligned to the shape of sequences it saw during training.

Accuracy is perfect for $d \leq 4$ and declines gradually at $d \geq 5$, consistent with floating-point drift over recursive application. The cell is a *structure-agnostic* operator function.

10 Decomposition: what TreeMoTE does and does not solve

The compositional-generalization problem decomposes into three components:

1. **Parsing:** mapping tokens to a computation graph.
2. **Local operator learning:** fitting the primitive per-node function.
3. **Recursive evaluation:** applying the learned function at every node.

Flat architectures must learn all three simultaneously. TreeMoTE receives the parse and shares a cell recursively; it is designed to address only component (3), leaving (1) external and (2) as a standard supervised-learning problem. We evaluate each boundary.

10.1 Operator learning at larger moduli ($p = 31$)

With 200K training sequences per depth at $p = 31$ and 60K training steps, TreeMoTE’s cell does not fully converge: training accuracy at $d = 2$ saturates near 0.50. The operator table $p^2 \times |\text{ops}| = 2883$ triples is large relative to our training budget, and partial cell learning limits total accuracy.

Table 13: $p = 31$, 200K samples per depth, 60K steps. Chance = $1/31 \approx 0.032$.

Arch	$d = 2$ (train)	$d = 3$ (train)	$d = 5$ (OOD)	$d = 7$ (OOD)
Transformer	0.068	0.060	0.046	0.034
TreeMoTE	0.498	0.372	0.164	0.104

Transformer fails to learn the $p = 31$ training distribution at all. TreeMoTE’s advantage persists ($5\times$ chance vs $1.4\times$ chance at $d = 5$), but neither architecture matches the $p = 7$ result. This is a cell-learning bottleneck, not a tree-recursion failure.

10.2 Value-space extrapolation

Training TreeMoTE with operand values restricted to $\{0, 1, 2, 3\}$ and testing on $\{0, \dots, 6\}$:

Table 14: Operand-value OOD, $p = 7$, curriculum $\{2, 3, 4, 5\}$, 20K steps.

Arch	$d = 2$	$d = 3$	$d = 4$	$d = 5$	$d = 6$	$d = 7$	$d = 8$
Transformer	0.322	0.254	0.234	0.200	0.160	0.156	0.182
TreeMoTE	0.454	0.324	0.272	0.276	0.240	0.220	0.226

TreeMoTE’s cell does not reliably interpolate to unseen operand values. At $d = 2$, TreeMoTE achieves 45% (above the 30% lower bound for pure memorization of all-seen-operand combinations, indicating partial interpolation), while Transformer achieves 32%. TreeMoTE retains a measurable architectural advantage (+4–13pp across depths) but does not solve value-space extrapolation. This is consistent with the decomposition: the cell is learned, not symbolically reasoned.

10.3 Scale-invariance of the StackMoTE α

Scaling StackMoTE across $d_{\text{model}} \in \{64, 128, 256\}$ and $L \in \{2, 4\}$ ($31\times$ parameter variation, 104K to 3.2M params), OOD accuracy at $d = 5$ remains 0.22 ± 0.01 while in-distribution accuracy at $d = 2$ scales from 0.41 to 0.85. Within this regime and architecture family, compositional-depth extrapolation cannot be purchased with parameters alone.

11 Related work and scoping

Recursive neural networks and tree-structured architectures predate us: Tree-LSTMs [5], recursive semantic composition [6], neural module networks [7], neural programmer-interpreters [8], neural stack machines [9, 10], stack-attention Transformers [11], and tree stack memory units [12] have all explored the idea that explicit structural inductive bias can help compositional learning. Petty et al. [3] document diminishing returns with depth for Transformer language models on COGS-style tasks. Kim and Linzen [2] established the canonical benchmark. Recent work on error accumulation in LLMs [4] uses $(1 - e)^n$ as a standard per-step decay baseline and argues it is the wrong form for long-form language generation.

Our contribution is not that structure helps—that is established—but that:

- Depth-decay admits a controlled two-parameter measurement via Eq. 1, whose parameters fingerprint architectures.
- α is causally tied to *dispatch discovery*: arch that need not learn dispatch exhibit $\alpha \rightarrow 0$.
- Compositional-generalization failure decomposes into three components (parsing, local learning, recursive evaluation), each addressable separately.
- The breakthrough is conditional and empirically falsifiable; we document where it holds ($p = 7$, sufficient coverage) and where it does not ($p = 31$ in our budget, operand-value OOD).

TreeMoTE applies only where the parse is given. For natural language and other tasks where the computation graph must be inferred from tokens, our work suggests the problem decomposes into parsing plus evaluation, with the evaluation bottleneck addressed by our approach and the parsing bottleneck left for future work.

12 Limitations and honest corrections

Explicit parse. TreeMoTE’s breakthrough requires balanced- or linear-RPN position conventions; the tree is recoverable from token positions. Natural language and free-form mathematical text require a parser, which we have not built.

Cell learning at larger moduli. At $p = 31$ and $p = 97$ (untested), the operator table is too large for our training budget. Scaling the cell, increasing data, and longer training are straightforward follow-ups but do not reduce the architectural claim of this work.

Null-model calibration. An earlier draft reported gaps against a uniform $1/p$ baseline. Empirical majority-class is ≈ 0.20 at $p = 7$ due to multiplication collapsing on zero. All numbers in this paper use rejection-sampled label-balanced data.

Single-seed variance. We report three seeds for the central claim. A preliminary single-seed configuration at $d_{\text{model}} = 128, L = 2$ showed an anomalous +16pp jump that did not replicate on extended training; we have flagged and removed such outliers.

Random-initialized structural prior. A randomly-initialized TreeMoTE gives $\approx 25\%$ test accuracy at $d = 5$ (above the 14.3% uniform chance), reflecting the architecture’s inductive bias before training. Training then converts this prior into the full 100% solution, a 75-percentage-point improvement. This is the *intended* behavior—architectural prior plus optimization—not a leak.

Train/test overlap at $d = 2$. With 3000 training and 500 test sequences at $d = 2$ (space size 3087), train/test overlap is $\approx 61\%$ by the pigeonhole. This is mathematically unavoidable for the smallest depth. For $d \geq 4$, overlap is zero; our out-of-distribution claims are at $d \geq 5$.

13 Conclusion

The compositional-depth decay of neural architectures on modular-arithmetic RPN is not an inherent property of the task but the cost of learning dispatch during forward pass. When the parse is given and a cell is shared recursively, the decay collapses: TreeMoTE reaches 100% accuracy at training depths and out-of-distribution depths alike, a mechanistic consequence of exactly learning the operator table and recursively applying the cell. Remaining failures—larger moduli, value-space extrapolation, and natural-language parsing—are separable problems with standard machine-learning paths forward.

A Two-parameter fit diagnostic

Over 28 runs with ≥ 5 depth evaluations, mean R^2 rises from 0.795 (one-parameter, floor = chance) to 0.826 (two-parameter, floor free). In 10/28 runs, $\Delta R^2 > 0.05$. Improvements concentrate in configurations where accuracy plateaus at higher depths—the signature of a non-zero floor.

B Scale sweep results

Table 15: StackMoTE (no-resid) scale sweep, curriculum $\{2, 3, 4\}$, 15K steps, seed 42.

(d_{model}, L)	Params	$d = 2$	$d = 3$	$d = 5$ (OOD)	$d = 6$ (OOD)
(64, 2)	104K	0.456	0.310	0.226	0.200
(64, 4)	207K	0.430	0.298	0.208	0.182
(128, 2)	409K	0.878	0.668	0.296	0.240
(128, 4)	815K	0.536	0.360	0.204	0.188
(256, 2)	1.6M	0.494	0.274	0.188	0.184
(256, 4)	3.2M	0.644	0.394	0.254	0.252

References

- [1] Lake, B. M., Baroni, M. (2018). *Generalization without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks*. ICML.
- [2] Kim, N., Linzen, T. (2020). *COGS: A Compositional Generalization Challenge Based on Semantic Interpretation*. EMNLP.
- [3] Petty, J., van Steenkiste, S., Linzen, T. (2025). *The Impact of Depth on Compositional Generalization in Transformer Language Models*. TMLR / arXiv:2310.19956.
- [4] Authors of arXiv:2505.24187 (2025). *Beyond Exponential Decay: Rethinking Error Accumulation in LLMs*. arXiv preprint.
- [5] Tai, K. S., Socher, R., Manning, C. D. (2015). *Improved Semantic Representations from Tree-Structured Long Short-Term Memory Networks*. ACL.

- [6] Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A. Y., Potts, C. (2013). *Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank*. EMNLP.
- [7] Andreas, J., Rohrbach, M., Darrell, T., Klein, D. (2016). *Neural Module Networks*. CVPR.
- [8] Reed, S., de Freitas, N. (2016). *Neural Programmer-Interpreters*. ICLR.
- [9] Joulin, A., Mikolov, T. (2015). *Inferring Algorithmic Patterns with Stack-Augmented Recurrent Nets*. NeurIPS.
- [10] Grefenstette, E., Hermann, K. M., Suleyman, M., Blunsom, P. (2015). *Learning to Transduce with Unbounded Memory*. NeurIPS.
- [11] DuSell, B., Chiang, D. (2024). *Stack Attention: Improving the Ability of Transformers to Model Hierarchical Patterns*. ICLR.
- [12] Authors of arXiv:1911.01545 (2019). *Tree Stack Memory Units for Compositional Generalization*. arXiv preprint.